

UCS503 Project Progress Report

Campus Honest & Transparent Feedback System — Technical Action Items (MOM 0817)

PROJECT TEAM	COURSE CODE	PLATFORM	COMPLETION STATUS
TechNova (3Q1A)	UCS503 Software Engineering	Thapar Institute (TIET)	5 of 5 Tasks (100% DONE)

Executive Summary & Task Completion Matrix

This report documents the end-to-end technical fulfillment of the 5 prioritized action items assigned in **MOM 0817** for the Campus Feedback System. All database models, end-user interactions, REST protocols, ACID transactions, and concurrency benchmarks have been successfully implemented and verified.

PRIORITY	ACTION ITEM DESCRIPTION	STATUS	IMPLEMENTED DELIVERABLE & ARTIFACTS
1 (Highest)	The DB with basic CRUD ops on each table	100% DONE	Normalized PostgreSQL schema across 11 relational tables with Prisma ORM CRUD services.
2	List out all interactions offered to end-user (EU)	100% DONE	Comprehensive user flow for Student (EU-S) and Faculty (EU-T) personas.
3	Formulate a rough protocol for each interaction	100% DONE	REST API contracts, Gateway routes (:8000), payload structures, and response codes.
4	Decompose interactions into atomic DB transactions	100% DONE	ACID transactions for review creation, cooldown checks, vote locking, and time-decay rating calculations.
5	Running the concurrency tests	100% DONE	Automated benchmark verifying race conditions, unique composite constraints, and zero deadlocks.

1. Database Schema & Basic CRUD Operations on Each Table

The system utilizes a relational PostgreSQL database hosted on Neon, accessed through Prisma ORM across our microservices architecture:

TABLE / MODEL	PRIMARY FUNCTION	CREATE (C)	READ (R)	UPDATE (U)	DELETE (D)
User	Auth & role classification	Signup flow	/auth/me, Login	isProfileComplete	Cascade delete
OtpVerification	Email OTP verification	sendOtp	Validate OTP query	Expiry renewal	Auto-cleanup
StudentProfile	Roll No, Name, Branch, Batch	upsert profile	By userId / Roll No	Update profile data	Cascade delete
TeacherProfile	Department, Title, Name	upsert profile	Faculty directory list	Update department/title	Cascade delete
CourseOffering	Course-to-batch mapping	Add course taught	Eligible teacher matching	Update course details	Offering delete
Review	Ratings, student identity, text	Create review	Teacher review feed	Moderation flag update	Cascade delete
ReviewVote	Helpful/Unhelpful review votes	Cast vote	Vote tally counts	Toggle UP / DOWN	Cascade delete
ReviewFlag	Abuse/spam reporting	Report review	Flagged review queue	Update status	Admin delete
ReviewReply	Threaded discussion replies	Create reply	Discussion list	Edit reply	Cascade delete
ReplyVote	Upvoting/downvoting replies	Cast reply vote	Reply vote tallies	Toggle UP / DOWN	Cascade delete
ProfileRating	Dual time-weighted analytics	Initial calculation	/ratings/:id	Auto-recalculation	Cascade delete

2. End-User (EU) Interactions Breakdown

A. Student Persona (EU-S)

- **Verified Authentication:** Strictly requires @thapar.edu email. Automatic keyword parsing (e.g. be23–be30) locks the role to STUDENT.
- **Student Onboarding:** Enters Full Name, 10-digit roll number, engineering branch, batch group (3Q11–3Q15, 2Q11–2Q15), and year of study.
- **Faculty Discovery:** Two distinct views:
 - *Eligible Teachers:* Automatically filtered to teachers registered for the student's active batch/branch.
 - *All Campus Faculty:* University-wide directory with real-time name & department search.
- **Transparent Review Submission:** Submits 1–5 star ratings and descriptive feedback. Student identity is transparently visible as Name (RollNumber) and email.
- **Anti-Spam 21-Day Cooldown:** Prevents review bombing by restricting students to 1 review per faculty member every 21 days.
- **Community Validation:** Casts helpful/unhelpful votes on reviews (strictly 1 vote per review).
- **Peer Discussion & Reply Voting:** Participates in threaded discussion under reviews and upvotes/downvotes peer replies.
- **Content Reporting:** Flags inappropriate reviews for administrative moderation.

B. Faculty / Teacher Persona (EU-T)

- **Verified Faculty Identification:** Non-student @thapar.edu emails are assigned the TEACHER role.
- **Faculty Profile & Course Mapping:** Registers full name, department, designation, and assigns courses taught to specific batch groups (e.g. UCS120 to 3Q11).
- **Time-Decayed Performance Analytics:**
 - **Overall Time-Decayed Rating:** Pedagogical decay weighting ($w_i = \frac{1}{1 + \text{age}/30}$).
 - **Current Rating:** Rolling 180-day recent semester rating.
- **Faculty Constructive Response:** Faculty members can reply directly to student reviews left on their courses to seek constructive suggestions or clarify lecture concepts.
- **Platform Integrity Guardrails:** Faculty accounts are restricted from reviewing other teachers or self-reviewing.

3. Formulated Interaction Protocols (REST API Contracts)

All client-server interactions are routed through an API Gateway (Port 8000) reverse-proxying requests to microservices:

INTERACTION	METHOD	GATEWAY ENDPOINT	PAYLOAD SUMMARY	RESPONSE CODE
Send OTP	POST	/api/auth/send-otp	{ email }	200 OK
Signup & Verify	POST	/api/auth/signup	{ email, password, otp }	201 Created
Student Onboarding	POST	/api/profiles/student	{ fullName, rollNumber, branch, batch, yearOfStudy }	200 OK
Teacher Onboarding	POST	/api/profiles/teacher	{ fullName, department, designation, offerings[] }	200 OK
Get Eligible Faculty	GET	/api/courses/eligible-teachers	Authorization: Bearer <token>	200 OK
Create Review	POST	/api/reviews	{ reviewer{}, reviewee{}, rating, reviewText, courseCode }	201 Created
Vote on Review	POST	/api/reviews/:id/vote	{ user: { userId }, vote: { type: "UP" "DOWN" } }	200 OK
Post Reply	POST	/api/reviews/:id/replies	{ authorId, authorRole, authorName, authorBadge, replyText }	201 Created
Vote on Reply	POST	/api/reviews/replies/:id/vote	{ user: { userId }, vote: { type: "UP" "DOWN" } }	200 OK
Get Teacher Ratings	GET	/api/ratings/:teacherId	None	200 OK

4. Decomposed Atomic DB Operations & ACID Transactions

Key operations are encapsulated in serialized Prisma database transactions to guarantee data consistency:

Review Submission & Rating Recalculation Transaction

```
// Atomic transaction enforcing 21-day cooldown check, creation, and rating recalculation
const review = await prisma.$transaction(async (tx) => {
  // 1. Serialize cooldown check
  const lastReview = await tx.review.findFirst({
    where: { reviewerId, revieweeId, createdAt: { gte: cooldownThreshold } },
    orderBy: { createdAt: "desc" }
  });

  if (lastReview) {
    throw new Error("21-day cooldown active. One review permitted every 21 days.");
  }

  // 2. Create the review record
  return await tx.review.create({
    data: { reviewerId, reviewerName, reviewerEmail, reviewerRollNo, reviewerBatch,
            reviewerBranch, revieweeId, courseCode, courseName, rating, reviewText }
  });
});

// 3. Trigger immediate mathematical recalculation of time-decay ratings
await calculateRating(revieweeId);
```

Mathematical Formulation of Dual Ratings:

- **Overall Time-Decay Rating:** $(R_{\text{overall}}) = \frac{\sum (r_i \cdot w_i)}{\sum w_i}$, where weight $(w_i = \frac{1}{1 + \text{ageInDays}/30})$.
- **Current Rating:** $(R_{\text{recent}}) = \frac{\sum (r_j \cdot w_j)}{\sum w_j}$ evaluated strictly for reviews where $(\text{ageInDays} \leq 180)$.

5. Concurrency & Stress Testing Benchmark Results

We executed an automated multi-threaded concurrency benchmark script (auth-service/test-concurrency.js) simulating high-volume simultaneous client transactions:

```
=====
CAMPUS FEEDBACK SYSTEM - ATOMIC & CONCURRENCY BENCHMARK TEST
=====
```

- ▶ [TEST 1] Concurrency on Review Creation (21-Day Cooldown Race Condition)
Total Parallel Requests: 5 simultaneous submissions
Successful Submissions: 1 (Expected: 1)
Blocked by Cooldown: 4 (Expected: 4)
✅ PASS: Cooldown safely serialized and prevented duplicate reviews under race condition.

- ▶ [TEST 2] High-Volume Concurrent Voting (Atomic Unique Constraint Validation)
Concurrent Voters: 20 parallel transactions
Execution Time: 2582ms
Recorded Votes in DB: 20 / 20
Data Consistency Check: 100% MATCH (ACID Compliant)
✅ PASS: Parallel votes processed with zero deadlocks or lost updates.

- ▶ [TEST 3] Duplicate Concurrent Votes from Same Student (Idempotency Check)
Simultaneous Attempts: 4 rapid parallel clicks
Distinct Votes in DB: 1 (Expected: 1)
✅ PASS: Unique composite key enforced 1-vote-per-student limit under race condition.

```
=====
ALL CONCURRENCY AND ATOMIC TRANSACTION TESTS PASSED! (100%)
=====
```

Conclusion: The system has achieved 100% compliance across all 5 action items of MOM 0817, maintaining high performance, platform integrity, transparent student identity, and faculty dialogue.